

Project Title: Developing and Evaluating a Timed Graph Neural Network for Chess Puzzle Solving Using Lichess Data

Project Overview

This project aims to explore the application of graph neural networks (GNNs) in chess, with a focus on incorporating temporal (timing) information to enhance puzzle-solving capabilities. Chess positions can be naturally represented as graphs, where pieces are nodes and possible moves or interactions are edges. By extending this to a "timed" graph network—potentially modeling move sequences with time-based features (e.g., time spent on moves or game clock data)—students will investigate whether such models can outperform traditional large language models (LLMs) in providing guidance for solving "mate in n moves" chess puzzles.

The project leverages the open-source Lichess dataset for training data generation and a held-out set of classic chess problems for validation. Training will utilize a custom library developed by Florence Wong and Ernesto Damiani.

The core research questions are: (1) For which values of n (depth of mate) does the timed graph network provide superior guidance compared to an LLM? (2) Does incorporating timing information improve the model's performance?

This project requires programming skills in Python, familiarity with deep learning frameworks (e.g., PyTorch or TensorFlow), and an interest in game AI. Estimated duration: 3 weeks, depending on team size (2-4 students recommended).

Background and Motivation

Chess has long been a benchmark for AI, from rule-based engines like Stockfish to neural network-based systems like AlphaZero. Read a summary on chess relevance to IA in <https://kustreview.com/capturing-style/>.

Graph neural networks are particularly promising for chess because the board state can be encoded as a graph: nodes represent squares or pieces with attributes (e.g., piece type, color, position), and edges capture relationships such as attacks, defenses, and mobility. A "timed" variant integrates temporal dynamics, such as move durations from real games, to model human-like decision-making under time pressure.

The Lichess platform hosts millions of games and puzzles, providing a rich dataset for training. Puzzles often involve tactical motifs that lead to mate in a fixed number of moves (n). By training a GNN on Lichess data, the model can learn to predict optimal moves or evaluate positions. Comparing this to an LLM (e.g., GPT-4 or a fine-tuned chess variant) highlights the differences.

Key challenges include:

- Integrating timing data meaningfully (e.g., as edge weights or node features).
- Ensuring fair comparison with LLMs, which might require prompting strategies.

Objectives

1. **Data Preparation:** Generate a training dataset from Lichess puzzles, incorporating timing information where available.
2. **Model Development:** Implement and train a timed graph neural network using the provided library.
3. **Validation Setup:** Curate a held-out set of "mate in n moves" problems and develop a pipeline for model guidance.
4. **Comparative Evaluation:** Assess the GNN's performance against an LLM on puzzle solving, focusing on n values where the GNN prevails.
5. **Ablation Study:** Analyze the impact of timing information through controlled experiments.
6. **Documentation and Insights:** Produce a report with findings, code, and potential extensions (e.g., real-time chess assistance).

Methodology

1. Data Acquisition and Preparation

- **Lichess Dataset:** Use the Lichess puzzle database (available as a CSV download from lichess.org/api#tag/Puzzles) and the elite games database (PGN format from database.lichess.org). The puzzle dataset includes over 2 million entries with fields like Puzzle ID, FEN (Forsyth-Edwards Notation) position, moves (in UCI format), rating, popularity, and themes (e.g., "mateIn2").
- **Training Set Generation:**
 - Filter Lichess data for positions leading to mate in 1-5 moves (common in puzzles).
 - Represent each position as a graph: Nodes for 64 squares (features: piece type, color); edges for legal moves, attacks, or pins.
 - Incorporate timing: For games, use move durations as temporal features (e.g., normalize time spent per move as a node/edge attribute). For puzzles (which lack inherent timing), augment with simulated times based on puzzle rating (higher rating = longer think time) or average times from similar Lichess games.
 - Label data: Target is the optimal move sequence or position evaluation (e.g., win probability or mate depth).
 - Split: 80% training, 10% validation (internal), 10% test (but hold out classic problems for final eval).
 - Tools: Use Python libraries like PyTorch Geometric for graph construction.
- **Held-Out Validation Set:** Source classic "mate in n moves" problems from public datasets, such as:
 - Chess.com's puzzle database or Kaggle's "Chess Puzzles" dataset (in PGN/FEN format).
 - Ensure format compatibility: FEN starting position, solution moves in UCI, and n ranging from 1 to 10 (to test depth limits).
 - Aim for 100-200 problems, stratified by n (e.g., 30 each for n=1 to 5, fewer for deeper n due to rarity).

2. Model Training

- **Timed Graph Network:** Use the library by Florence Wong and Ernesto Damiani and training utilities.
 - Output: Move predictions (policy head) or value estimation (e.g., probability of mate in n).
 - Training: Supervised learning on Lichess data, optimizing for move accuracy or mate detection. Use GPU for efficiency; epochs ~50-100, batch size 32-128.
 - Hyperparameters: Tune learning rate ($1e-3$), layers (3-5), hidden dims (128-256).

3. Guidance Generation and Comparison

- **GNN Guidance:** For a puzzle, input the position graph; output suggested moves, confidence scores
- **LLM Baseline:** Use an open LLM such as Llama-2 or GPT via an API. Prompt: "Given this FEN: [FEN], find mate in n moves. Explain step-by-step." Limit to zero-shot or few-shot to simulate guidance without fine-tuning.
- **Comparison Protocol:**
 - Metrics: Accuracy (correct mate sequence), efficiency (time to solution), guidance quality (human-rated clarity, e.g., via rubric: 1-5 for correctness, conciseness).
 - Run on held-out set: For each puzzle, generate guidance from both models, and compare outputs.
 - Human evaluation: Students or experts rate which guidance is better (blind test).

4. Research Questions and Experiments

- **Values of n where GNN Prevails:**
 - Hypothesis: GNN is better for small n (1-3, where graph structure captures tactics directly) vs. LLM for larger n (4-10, where language-based reasoning shines).
 - Experiment: Stratify results by n ; compute win rates (GNN vs. LLM) using paired t-tests or McNemar's test.
 - Plot: Bar chart of accuracy by n .
- **Impact of Timing Information:**
 - Ablation: Train two models—one with timing features, one without.
 - Metrics: Compare performance delta on timed vs. untimed puzzles (simulate untimed by removing features).
 - Hypothesis: Timing helps in puzzles mimicking blitz games (short think times), improving move prediction by modeling urgency.

Resources Required

- **Compute:** Access to a GPU (e.g., Google Colab or university cluster) for training.
- **Software:** Python 3+, libraries: chess, torch, torch-geometric, custom Wong-Ernesto library (provided).
- **Data:** Download Lichess puzzles (~1GB CSV) and games (~TB, subsample to 100k games).

Expected Outcomes and Extensions

- Deliverables: Code repo, trained model, evaluation report, visualizations (e.g., confusion matrices by n).
- Insights: Quantify GNN advantages (e.g., "GNN prevails for $n \leq 3$ with 15% higher accuracy") and timing benefits (e.g., "10% uplift in blitz scenarios").
- Extensions: Deploy as a web app for chess training; integrate with Stockfish for hybrid solving; explore multi-modal inputs (e.g., board images).

This project fosters skills in AI research, from data to deployment, and contributes to understanding timed decision-making in games. Students should collaborate via GitHub and meet weekly for progress updates.